Database Systems and Security (7CI022)

Group 4

(Solagbade Akande (2417206), Sarah Oloko (2437410), & Chigozie Obiagazie(2427322))

Table of Contents

Introduction

CSS Car Manufacturing and Distribution Company is a company engaged in the manufacturing and sale of vehicles. The business operates under a dual model:

- **B2C** (Business-to-Consumer)**:** Direct sales to customers.
- **B2B** (Business-to-Business)**:** Bulk sales to dealers.

It also provides servicing for retail customers and dealers. Salespersons manage orders across regions.

The company aims to optimize order processing, servicing, and sales management through an effective database system.

Modelling – Part 1

1.Business Rules

CSS sells a range of cars up to 100 models which are priced from £10-100K. CSS is open Monday to Friday 9-5pm to all customers and the information below explains the information needs and data involved to create a database for the business.

The following business rules define the structure and operations of CSS Car Manufacturing and Distribution Company:

Sales & Order Management:

1. **Retail customers and dealers** can both place orders.
2. **Orders** contain one or more vehicles.
3. Each **order** is assigned to a **salesperson**, who handles the transaction.
4. Every **order is linked to a specific region**, determining where it is processed.

Vehicle Servicing:

5. Retail customers and dealers can request vehicle servicing.
6. Servicing is always linked to a **specific vehicle**.
7. A vehicle **may undergo multiple servicing sessions over time**.

Dealer & Retail Customer Relationships:

8. **Dealers purchase vehicles in bulk**, while customers typically buy in smaller quantities.

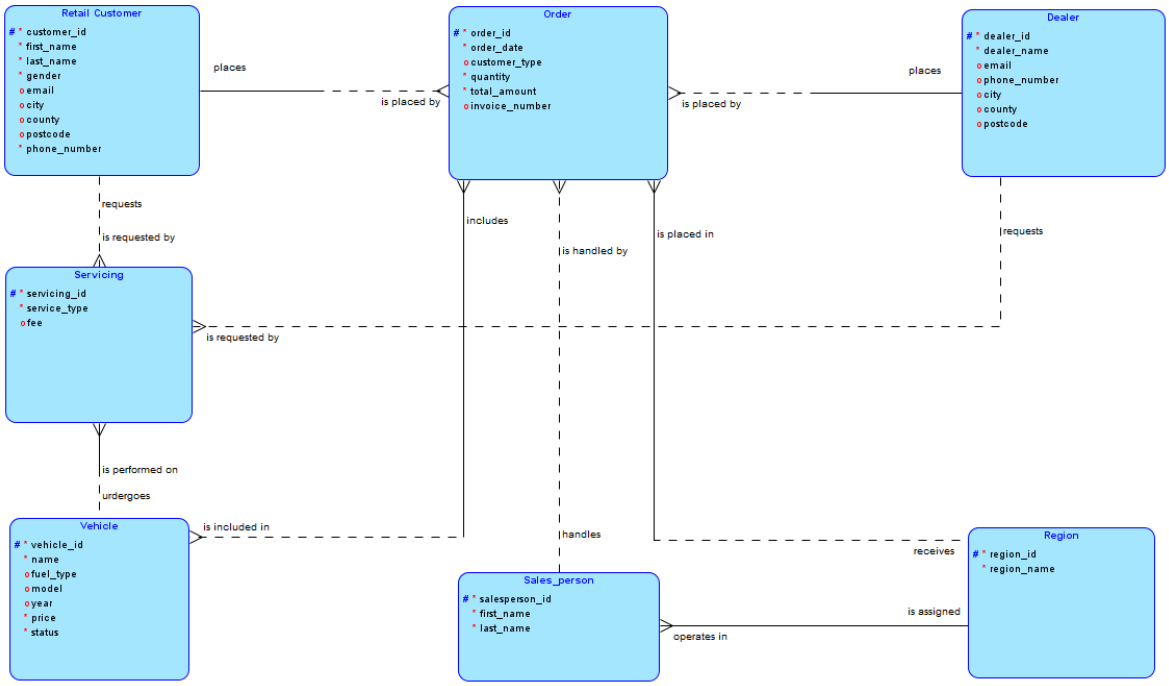
Salesperson & Regional Operations:

9. Each **salesperson is assigned to a specific region**.
10. A **salesperson may handle multiple orders**, but each order is handled by **only one salesperson**.
11. **Orders are placed within a region**, meaning each order must be linked to one region.
12. A **region can have multiple salespersons**, but a salesperson belongs to only **one region**.

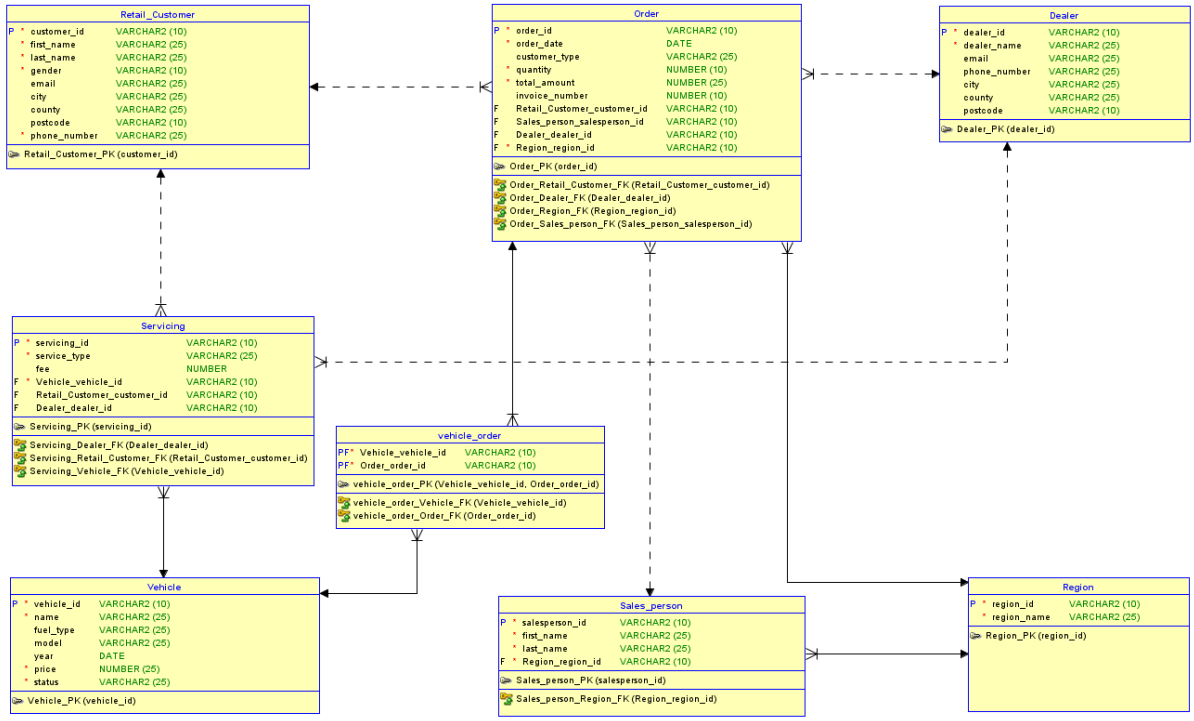
These business rules **ensure consistency in order management, servicing, and dealership operations**.

2. ER Diagram Discussion

Logical Model



Entity Relational Model



The logical model was designed using Crow's Foot Notation to structure the database, defining entities, attributes, and relationships. Entities such as Retail Customer, Dealer, Order, Vehicle, Servicing, Salesperson, and Region were identified, with relationships described using verbs (e.g., a Customer ***places*** an Order).

The cardinalities (e.g One-to-Many, Many-to-Many...) were defined. The solid lines indicate mandatory relationships (e.g., an Order must be placed in a Region), while dashed lines indicate optional relationships (e.g., a Vehicle may or may not be included in an Order if unsold).

The logical model was then engineered into the relational model, explicitly showing primary and foreign keys which enforce referential integrity. For example, the Order table references customer_id, salesperson_id, dealer_id, and region_id as foreign keys.

A junction table, *vehicle_order*, was introduced to manage the many-to-many relationship between Order and Vehicle.

This structured approach ensures a clear schema, enforcing business rules and enabling efficient data retrieval.

Entities and Their Relationships

1. Retail Customer

- Source**: Retail Customer** → Target**: Order** *(1:M)*
  A Retail **Customer must place one or more Orders**, meaning the relationship is **one-to-many (1:M) from Retail Customer to Order**. *(To be registered as a Customer in our company, an entity must have placed at least one Order.)* However, an **Order may or may not be placed by a Retail Customer**, since some Orders may be placed by Dealers instead.
- Source**: Retail Customer** → Target**: Servicing** *(1:M)*
  A Retail **Customer may request one or more Servicing instances**, meaning this relationship is **one-to-many (1:M)**. However, **Servicing may or may not be requested by a Retail Customer**, as it could also be requested by a Dealer.

2. Dealer

- Source**: Dealer** → Target**: Order** *(1:M)*
  A **Dealer must place one or more Orders**, meaning the relationship is **one-to-many (1:M) from Dealer to Order**. An **Order may or may not be placed by a Dealer**, since some Orders are placed by Retail Customers instead.
- Source**: Dealer** → Target**: Servicing** *(1:M)*
  A **Dealer may request one or more Servicing instances**, meaning this relationship is **one-to-many (1:M)**. **Servicing may or may not be requested by a Dealer**, as it could also be requested by a Retail Customer.

3. Order

- Source**: Order** → Target**: Vehicle** *(M:N)*
  This is a **many-to-many (M:M) from Order to Vehicle**.
  Each **Order must include one or more Vehicles**
  Each **Vehicle may be part of one or more orders -** 'may' is used because some Vehicles might still be in inventory and unsold.
  A **vehicle_order** table has been created to manage this relationship, as it is a many-to-many relationship between **Order** and **Vehicle.**
  The **vehicle_order** table stores pairs of **order_id** and **vehicle_id** to represent which vehicles are included in each order.

- Source**: Order** → Target**: Salesperson** *(M:1)*
  This is a many-to-one (M:1) relationship from Order to Sales
  Each **order must be handled by one and only one salesperson**
  Each salesperson **may handle one or more orders - 'may' is used because a salesperson may not have had an opportunity to handle an order yet.**

- Source**: Order** → Target**: Region** *(M:1)*
  This is a many-to-one (M:1) relationship from Order to Region
  Each order must be placed in one and only one region
  Each region may receive at least one or more orders

4. Servicing

- Source**: Servicing** → Target**: Vehicle** *(M:1)*
  This is a many-to-one (M:1) relationship from Servicing to Vehicle
  Each servicing instance **must** have been performed on one and only one vehicle
  Each vehicle **may** undergo one or more servicing instances

5. Region

- Source: **Region** → Target: **Salesperson** (1:M)
  This is a one-to-many (1:M) relationship from Region to Salesperson
  Each region must have one or more salespersons
  Each salesperson must belong to one and only

3.Normalisation

Normalisation is important in good database design as it helps eliminate redundancies, multivalued attributes, and composite attributes which ensures data consistency and integrity. The diagrams below show our unnormalised and normalised tables that our entities are based on and the process which we took to get from unnormalised to 2NF tables.

Unnormalised Table

Please note this table shows just a snapshot of the attributes from our table.

salesperson_id	first_name	last_name	region_name	customer_id	customer_first_name	customer_last_name	customer_email	customer_phone_number	customer_gender	customer_address	customer_city	customer_county	customer_postcode
SA101	Ade	Olulode	London	C101	James	Stafford	jamesstaff@gmail.com	0	m	64 Green Street	London	Greater London	SW1 5XY
			London	#	James	Stafford	jamesstaff@gmail.com	0		64 Green Street	London	Greater London	
SA102	Jasmine	Little	Manchester	C102	Abena	Ocloo	lizzyjones@gmail.com	44743219215	f	12 Yellow Street	London	Greater London	
SA103	Ikenna	Adichie	Manchester	-	-	-	-	-	-	-	-	-	-
SA102	Jasmine	Little	Manchester	C102	-	-	-	-	-	-	-	-	-

This table above is unnormalised because it contains **duplicate data, missing values, and multi-valued attributes**, making it inefficient for structured database management.

There are repeated groups in this table such as the customer James Stafford who appears twice leading to unnecessary duplicates. There are multiple-values in cells in this table, for example for the salesperson Ikenna Adichie and Abena Ocloo which can be a problem in the future when filtering and sorting out data. There is also missing and inconsistent data in this table where there are # and - values which are placeholders for missing data.

1NF Table

Please note this table only shows a snapshot of the attributes from our table.

salesperson_id	first_name	last_name	region_name	customer_id	customer_first_name	customer_last_	customer_email	customer_phone_number	customer_gender	customer_city	customer_coun	customer_posti	date_of_birth
SA101	Ade	Olulode	London	C101	James	Stafford	jamesstaff@gmail.c	44712098654	m	London	Greater London	SW1 5XY	16/04/1981
SA102	Jasmine	Little	Manchester	C102	Abena	Ocloo	lizzyjones@gmail.c	44743219215	f	London	Greater London	M1 1AE	10/01/1990
SA103	Ikenna	Adichie	Manchester	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
SA102	Jasmine	Little	Manchester	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

dealer_id	dealer_name	dealer_email	dealer_city	dealer_county	dealer_postcode	dealer_phone_number	vehicle_id	vehicle_name	vehicle_type	vehicle_model	vehicle_price	vehicle_year	fuel_type	vehicle_status
NULL	NULL	NULL	NULL	NULL	NULL	NULL	HD001	CSS	CSS1	Model 1	£6,000.00	2024	electric	Available
NULL	NULL	NULL	NULL	NULL	NULL	NULL	HD002	CSS	CSS2	Model 2	£10,000.00	2025	diesel	Available
D101	Cars Plus	info@carsplus.com	Leeds,	West Yorkshire	BD1 11NE	44732987324	HD003	CSS	CSS3	Model 3	£15,000.00	2024	electric	Available
D101	Cars Plus	info@carsplus.com	Leeds	West Yorkshire	BD1 11NE	44732987324	HD004	CSS	CSS2	Model 2	£10,000.00	2025	petrol	Available

The table above is now a First Normal Form (1NF) table because all duplicates, missing values and multi-valued attributes have been removed. The null values represent data that does not need to be present such as row 4 where the order is with a dealer and not a customer so customer attributes are not relevant. This structure allows for **better data integrity, easier querying, and improved efficiency in database management**.

The table above shows NULL in some rows due to us having two different types of customers. B2C(retail customers) and B2B(car dealers). When a retail customer purchases a car, we do not need details from a dealer and when a dealer purchases a car, we do not need details from retails customers in the table.

2NF Table

Sales Person Table

Primary Key	Foreign Key

salesperson_id	first_name	last_name	region_id
SA001	Ade	Olulode	L001
SA002	Jasmine	Little	M002
SA003	Ikenna	Adichie	M002

Retail Customer Table

customer_id	customer_first_name	customer_last_name	customer_email	customer_phone_number	customer_gender	customer_city	customer_county	customer_postcode	date_of_birth
C001	James	Stafford	jamesstaff@gmail.com	44712098654	m	London	Greater London	SW1 5XY	16/04/1981
C002	Abena	Ocloo	abenaocloo@gmail.com	44743219215	f	London	Greater London	M1 1AE	10/01/1990

Dealer Table

dealer_id	dealer_name	dealer_email	dealer_city	dealer_county	dealer_postcode	dealer_phone_number
D001	Cars Plus	info@carsplus.com	Leeds	West Yorkshire	BD1 11NE	44732987324

Vehicle Table

vehicle_id	vehicle_name	vehicle_type	vehicle_model	vehicle_price	vehicle_year	fuel_type	vehicle_status
HD001	CSS	CSS1	Model 1	£6,000.00	2024	electric	Available
HD002	CSS	CSS2	Model 2	£10,000.00	2025	diesel	Available
HD003	CSS	CSS3	Model 3	£15,000.00	2024	electric	Available
HD004	CSS	CSS2	Model 2	£10,000.00	2025	petrol	Available

Order Table

order_id	order_date	vehicle id	customer_id	dealer_id	customer type	quantity	total_amount	invoice_number	salesperson_id	region_id
1	4/2/2024	HD001	C001	NULL	public customer	1	£6,000	INV001	SA001	L001
2	3/5/25	HD002	C002	NULL	public customer	1	£10,000	INV002	SA002	M001
3	9/11/24	HD003	NULL	D001	car dealer	10	£150,000	INV003	SA003	M001
4	6/2/25	HD004	NULL	D002	car dealer	10	£100,000	INV004	SA002	M001

Servicing Table

servicing_id	vehicle_id	customer_id	dealer_id	service_type	customer_type	fee
S001	CSS1	C001	NULL	tyre change	public customer	£300

Region Table

region_id	region_name
L001	London
M001	Manchester

The table above is the second normal form (2NF). This 2NF table identifies the primary keys, removes partial dependencies and creates new tables based on this. All attributes need to depend on the primary key which is where the new tables have been formed in 2NF. The dark green indicates a primary key and the light green indicates a foreign key.

The new tables formed are

- **Salesperson Table** (Contains salesperson details without region name)
- **Retail Customer Table** (Stores individual/retail customer details separately)
- **Car Dealer Table** (Keeps dealer information)
- **Vehicle Table** (Stores unique vehicle details)
- **Order Table** (Links orders with customers, salespersons, and vehicles)
- **Servicing Table** (Keeps servicing information)
- **Region Table** (Links region_id with region_name)

These new tables have reduced the data redundancies and increased the data integrity. 2NF is the final step for our attributes due to there not being any transitive attributes in our 2NF table. Both partial dependencies and transitive dependencies have been removed in 2NF.

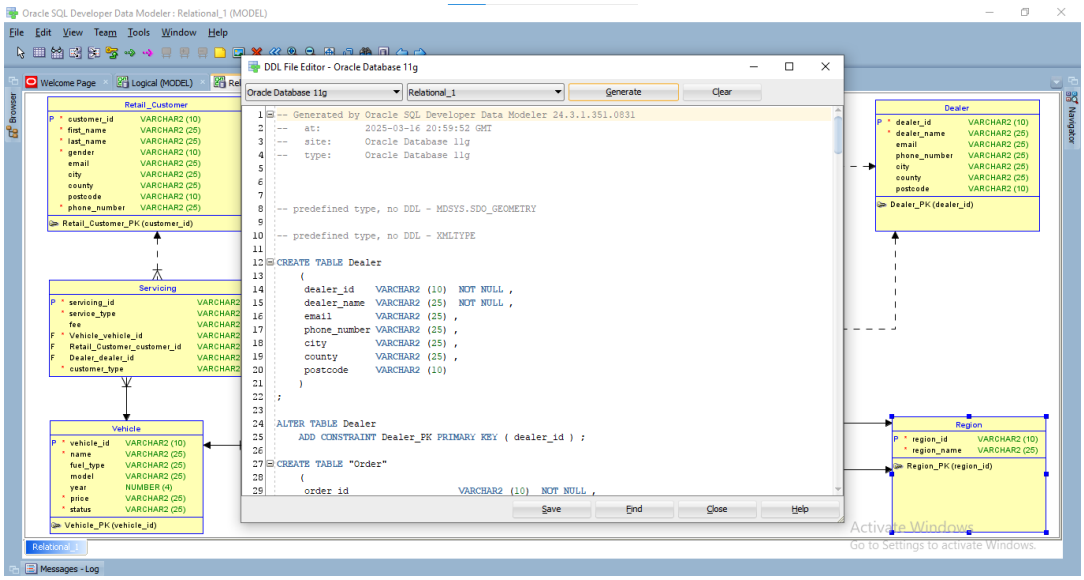
Implementation – Part 2

4.Database Implementation

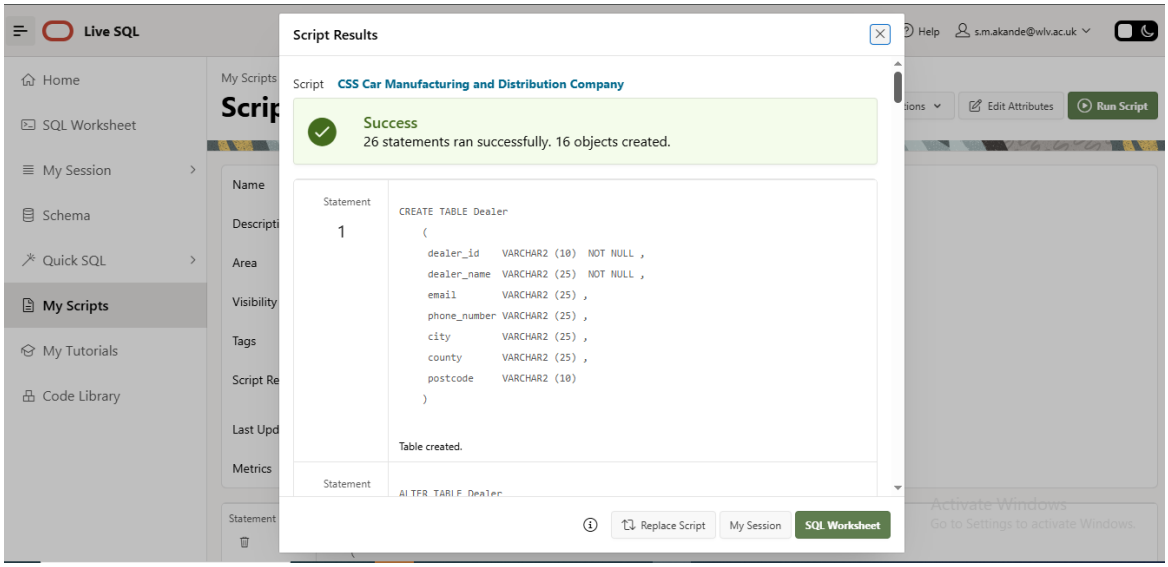
This section focuses on database implementation and the detailed steps required to implement our database.

Steps Taken to Create and Insert Data into Our Database

The first step was to create a DDL script from our Entity Relationship Model in Oracle as shown below. To view the full DDL script please refer to the appendix.



Once the DDL script was created it was uploaded to LiveSQL.



The tables below were created for our database as shown below. Once the tables were created, data was inserted into our database using the SQL commands show in in the screenshots below.

An intermediary table, **vehicle_order**, was automatically generated to resolve the **many-to-many (M:N) relationship** between **Vehicle** and **Order**. This table serves as a **bridge entity**, ensuring that each vehicle can be associated with multiple orders, and each order can

include multiple vehicles. It maintains referential integrity by storing **foreign keys** from both **Vehicle** and **Order**, thereby normalizing the database structure and preventing redundancy.

Table Name: Retail_Customer

Create Table Statement:

```
CREATE TABLE Retail_Customer
    (
     customer_id  VARCHAR2 (10)  NOT NULL ,
     first_name   VARCHAR2 (25)  NOT NULL ,
     last_name    VARCHAR2 (25)  NOT NULL ,
     gender       VARCHAR2 (10)  NOT NULL ,
     email        VARCHAR2 (25) ,
     city         VARCHAR2 (25) ,
     county       VARCHAR2 (25) ,
     postcode     VARCHAR2 (10) ,
     phone_number VARCHAR2 (25)  NOT NULL
    )
;

ALTER TABLE Retail_Customer
    ADD CONSTRAINT Retail_Customer_PK PRIMARY KEY ( customer_id ) ;
```

Insert Data Statement:

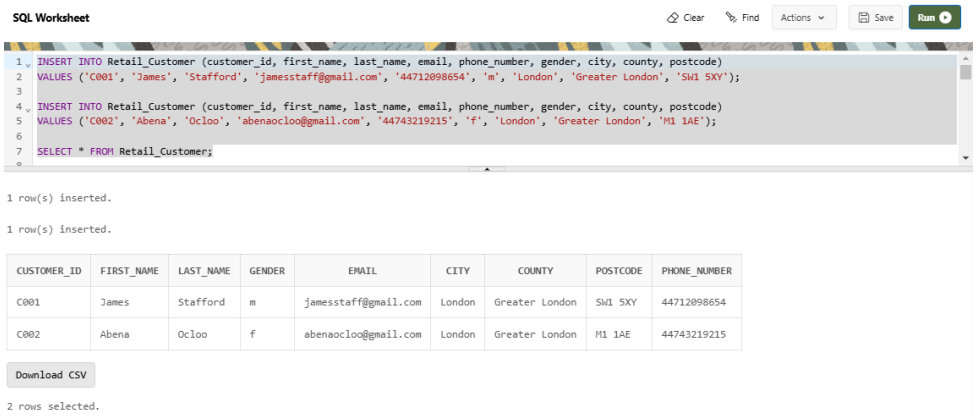


Table Name: Dealer

Create Table Statement:

```sql
CREATE TABLE Dealer
    (
     dealer_id    VARCHAR2 (10)  NOT NULL ,
     dealer_name  VARCHAR2 (25)  NOT NULL ,
     email        VARCHAR2 (25) ,
     phone_number VARCHAR2 (25) ,
     city         VARCHAR2 (25) ,
     county       VARCHAR2 (25) ,
     postcode     VARCHAR2 (10)
    )
;

ALTER TABLE Dealer
    ADD CONSTRAINT Dealer_PK PRIMARY KEY ( dealer_id ) ;
```

Insert Data Statements:

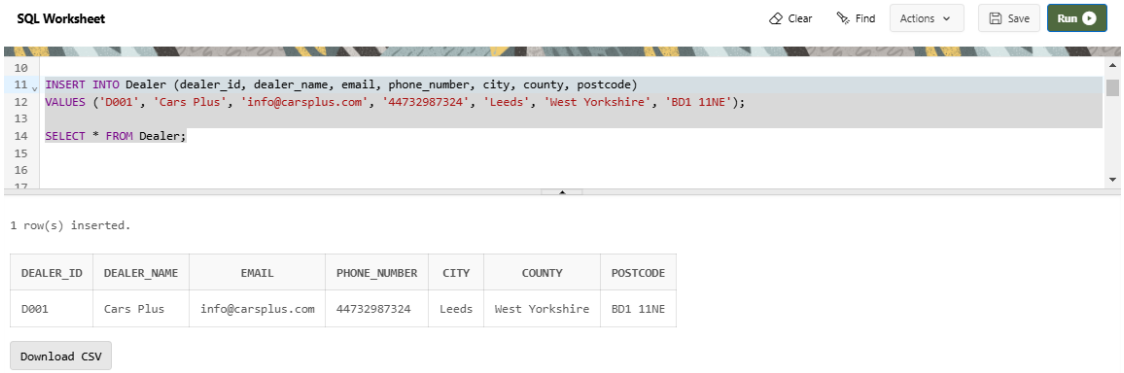


Table Name: Vehicle

Create Table Statement:

```sql
CREATE TABLE Vehicle
    (
     vehicle_id VARCHAR2 (10)  NOT NULL ,
     name       VARCHAR2 (25)  NOT NULL ,
     fuel_type  VARCHAR2 (25) ,
     model      VARCHAR2 (25) ,
     year       NUMBER (4) ,
     price      VARCHAR2 (25)  NOT NULL ,
     status     VARCHAR2 (25)  NOT NULL
    )
;

ALTER TABLE Vehicle
    ADD CONSTRAINT Vehicle_PK PRIMARY KEY ( vehicle_id ) ;
```

Insert Data Statements:

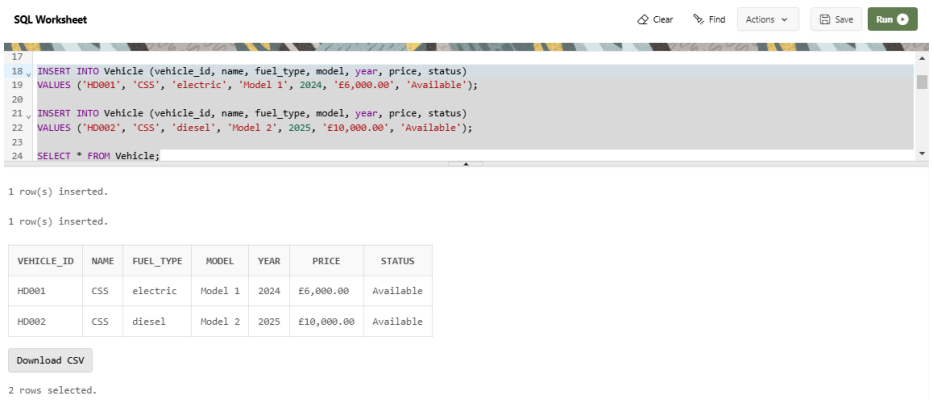


Table Name: Region

Create Table Statement:

```
CREATE TABLE Region
    (
     region_id   VARCHAR2 (10)  NOT NULL ,
     region_name VARCHAR2 (25)  NOT NULL
    )
;

ALTER TABLE Region
    ADD CONSTRAINT Region_PK PRIMARY KEY ( region_id ) ;
```

Insert Data Statements:

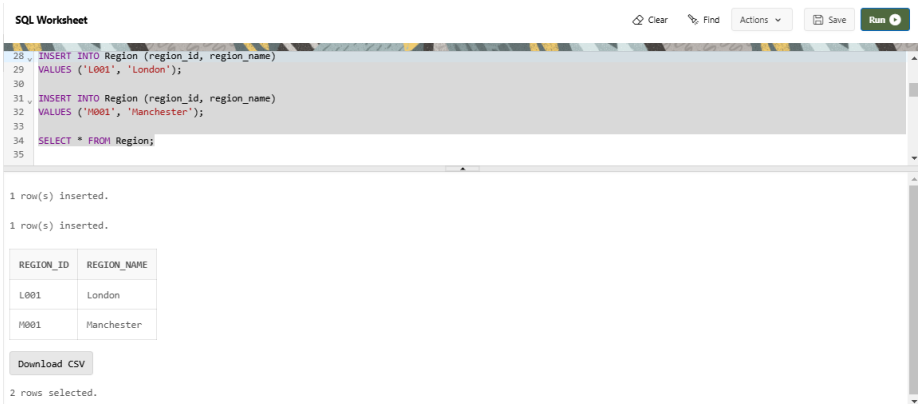


Table Name: Sales_person

Create Table Statement:

```
CREATE TABLE Sales_person
    (
     salesperson_id   VARCHAR2 (10)  NOT NULL ,
     first_name       VARCHAR2 (25)  NOT NULL ,
     last_name        VARCHAR2 (25)  NOT NULL ,
     Region_region_id VARCHAR2 (10)  NOT NULL
    )
;

ALTER TABLE Sales_person
    ADD CONSTRAINT Sales_person_PK PRIMARY KEY ( salesperson_id ) ;
```

Insert Data Statements:

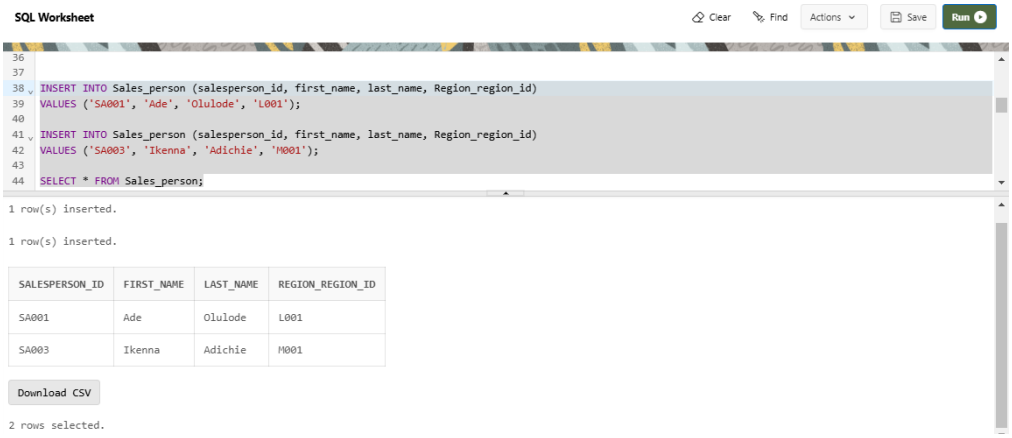


Table Name: ORDER

Create Table Statement:

```
CREATE TABLE "Order"
    (
     order_id                    VARCHAR2 (10)  NOT NULL ,
     order_date                  DATE  NOT NULL ,
     customer_type               VARCHAR2 (25) ,
     quantity                    NUMBER (10)  NOT NULL ,
     total_amount                VARCHAR2 (25)  NOT NULL ,
     invoice_number              VARCHAR2 (25) ,
     Retail_Customer_customer_id VARCHAR2 (10) ,
     Sales_person_salesperson_id VARCHAR2 (10) ,
     Dealer_dealer_id            VARCHAR2 (10) ,
     Region_region_id            VARCHAR2 (10)  NOT NULL
    )
;

ALTER TABLE "Order"
    ADD CONSTRAINT Order_PK PRIMARY KEY ( order_id ) ;
```

Insert Data Statements:

Table Name: vehicle_order

Create Table Statement:

```
CREATE TABLE vehicle_order
    (
     Vehicle_vehicle_id VARCHAR2 (10)  NOT NULL ,
     Order_order_id     VARCHAR2 (10)  NOT NULL
    )
;

ALTER TABLE vehicle_order
    ADD CONSTRAINT vehicle_order_PK PRIMARY KEY ( Vehicle_vehicle_id, Order_order_id ) ;
```

Insert Data Statements:

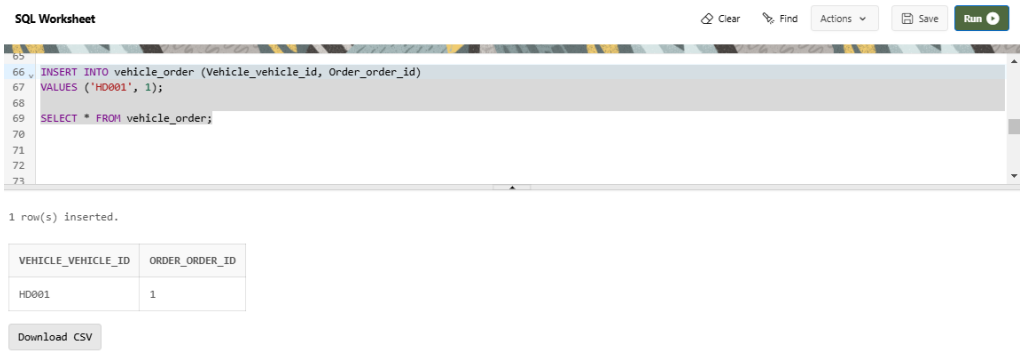


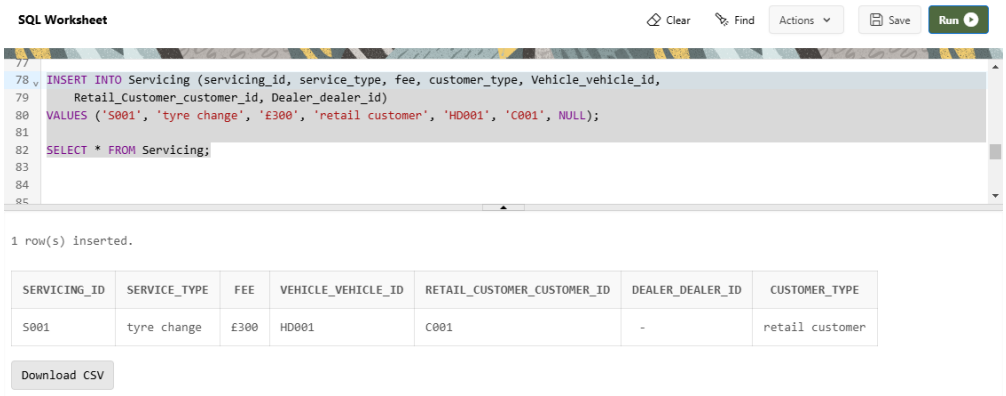
Table Name: Servicing

Create Table Statement:

```
CREATE TABLE Servicing
    (
     servicing_id                VARCHAR2 (10)  NOT NULL ,
     service_type                VARCHAR2 (25)  NOT NULL ,
     fee                         VARCHAR2 (25) ,
     Vehicle_vehicle_id          VARCHAR2 (10)  NOT NULL ,
     Retail_Customer_customer_id VARCHAR2 (10) ,
     Dealer_dealer_id            VARCHAR2 (10) ,
     customer_type               VARCHAR2 (25)  NOT NULL
    )
;

ALTER TABLE Servicing
    ADD CONSTRAINT Servicing_PK PRIMARY KEY ( servicing_id ) ;
```

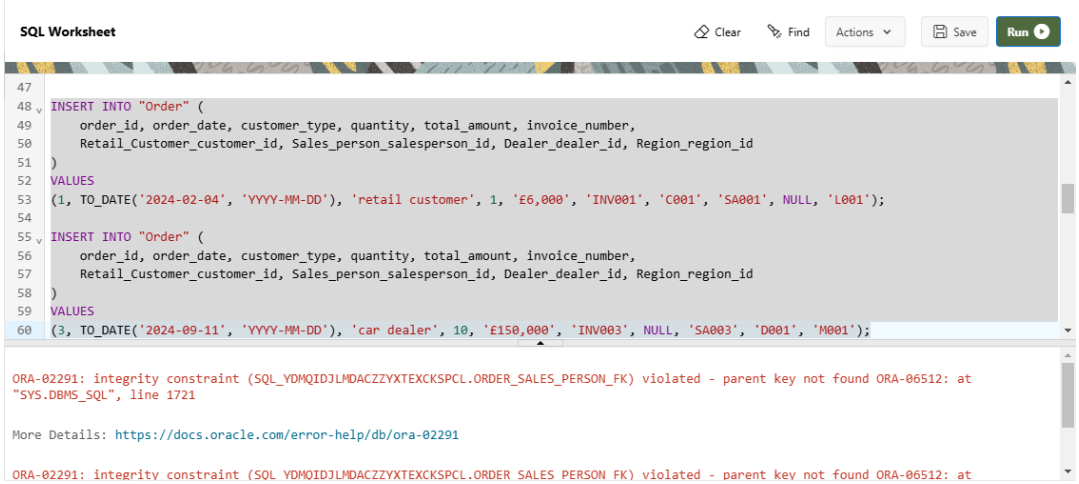
Insert Data Statements:



Ensuring Referential Integrity in Data Insertion

To maintain referential integrity, data had to be inserted in a structured sequence, as some tables depend on others. For example, the **Order** table references **Dealer**, **Retail_Customer**, **Vehicle**, **Region**, and **Sales_Person**, meaning these parent records must exist before inserting an order.

Failure to follow this would result in **ORA-02291: Integrity Constraint Violated - Parent Key Not Found**, as seen in the screenshot below. This occurs when an **order** references a **region** or **customer** that has not yet been inserted, violating the foreign key constraint.

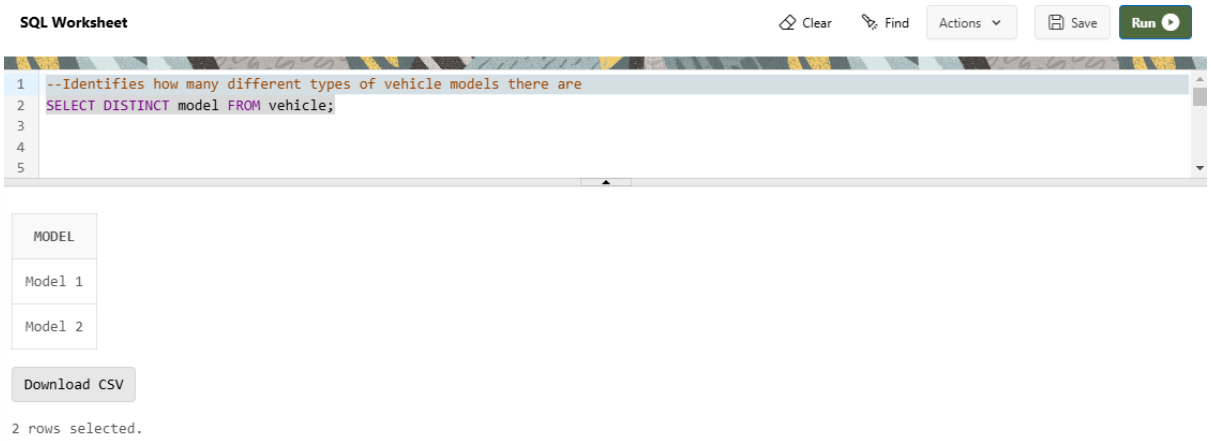


SQL Queries to Retrieve Data

These are the queries used to retrieve the data stored in our database.

Query 1

Query 1 is used to identify how many different types of models there are. For this the SELECT DISTINCT statement is used.

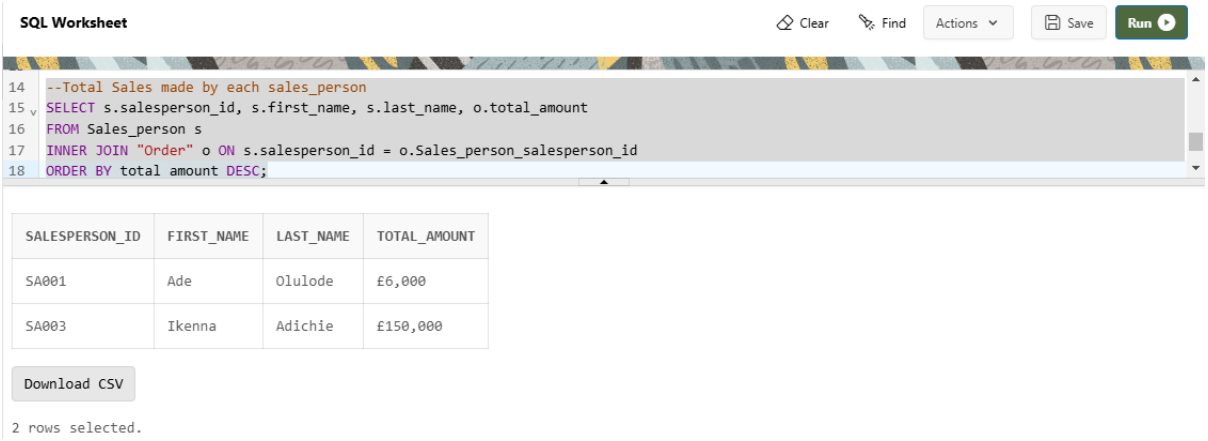


Query 2

Query 2 is a SELECT* FROM statement used to retrieve data about every salesperson based in the M001 region (Manchester).

Query 3

Query 3 uses a join(AND) operator that queries data from 2 or more tables. For this SQL query we retrieved total sales (total amount) by each salesperson in descending order, which took data from the sales_person table and the order table.



Query 4

Query 4 retrieves order details along with customer and vehicle information. It **joins** the **Order**, **Retail_Customer**, **Vehicle_order**, and **Vehicle** tables to display order ID, date, customer info, and the associated vehicle's name, model and fuel type

5. Security, Integrity and Ethics

Security in Database Management and Data Governance

Database security protects data from unauthorized access, breaches, and cyber-attacks, ensuring confidentiality, availability, and integrity. Key measures like authentication (verifying user identity), access control (restricting data access), encryption (securing data), and intrusion detection (identifying unauthorized access) are essential for safeguarding sensitive data. Despite technological advances, risks like SQL injection, ransomware, and vulnerabilities in cloud databases continue to challenge organizations. These threats require strong governance frameworks to mitigate risks, such as multi-tenancy and data sovereignty (Halfond et al., 2006).

Data Integrity in Database Management and Governance

Data integrity ensures accuracy, consistency, and reliability across databases. Violations can occur due to hardware failures, cyber-attacks, or human errors (Codd, 1970). Key principles such as entity integrity, referential integrity, and transaction integrity maintain the validity and consistency of data (Haerder & Reuter, 1983). Threats to integrity include human mistakes, cyber threats, and system breakdowns. Techniques like normalization (organizing data), validation checks (ensuring accuracy), and audit trails (tracking changes) help preserve data integrity and provide traceability for modifications.

Ethical Considerations in Database Governance

Ethical challenges in database governance revolve around privacy, fairness, transparency, and accountability. Breaches in ethical standards can damage trust and lead to legal consequences (Floridi, 2013). Issues such as privacy and consent, data ownership, and biases in automated decision-making systems are key concerns (O'Neil, 2016).
To mitigate privacy risks, the principle of data minimization ensures that only the data necessary for a specific purpose is collected and stored. This practice helps reduce unnecessary exposure and enhances data privacy (Richards & King, 2014). Also, Ethical frameworks, such as FIPPs, OECD Privacy Guidelines, and the GDPR, guide organizations in responsibly handling data (European Commission, 2018).

Security, Integrity, and Ethical Aspects in Our Vehicle Manufacturing Database

Our Vehicle Manufacturing Database is essential for managing sensitive data within our organization, supporting key business decisions and operations. As the reliance on this database grows, ensuring robust security, integrity, and ethical compliance in data governance is vital. This ensures the protection, reliability, and responsible use of the data stored within. In this context, we will explore the security, integrity, and ethical considerations relevant to managing our vehicle manufacturing database.

Security in Database Governance

Database security is crucial in protecting data from unauthorized access, breaches, and cyber-attacks. In our vehicle manufacturing database, sensitive customer and transactional data requires the implementation of various security measures:

• Access Control and Authentication: We will implement role-based access control (RBAC) to ensure that only authorized users (customers, dealers, salespersons) can access specific data. This will minimize the risk of unauthorized access and insider threats (Sandhu et al., 1996).
• Data Encryption: To protect sensitive customer data and financial transactions, we will apply AES-256 encryption for data at rest and during transit, ensuring its protection from cyberattacks (Stallings, 2019).

• Intrusion Detection Systems (IDS): These systems will actively monitor for any suspicious activities within our database, providing real-time detection and prevention of potential threats.
• Audit Trails: Maintaining a comprehensive log of all database transactions will help us detect and trace unauthorized changes, ensuring transparency and accountability.
• Backup and Disaster Recovery: We will implement regular backups and automated failover systems to guarantee recovery of data in the event of a failure or cyberattack.
Despite the security measures, ongoing challenges such as SQL injection and vulnerabilities in cloud-based DBMS require us to stay proactive with security governance (Halfond et al., 2006).

Data Integrity in Database Governance
Data integrity ensures that our database remains accurate, consistent, and reliable. We will focus on preventing duplication, incorrect entries, or inconsistencies, as these can disrupt business operations (Codd, 1970).
• Referential Integrity: We will implement foreign key constraints to ensure that relationships between tables (e.g., between vehicles and servicing records) remain consistent and free from orphaned records (Connolly & Begg, 2015).
• Transaction Integrity: We will ensure ACID compliance in all transactions, ensuring data consistency and preventing partial updates (Haerder & Reuter, 1983).
• Data Validation: Implementing validation rules will guarantee that data entered into our database meets necessary requirements, such as correct formatting and completion of mandatory fields.
• Duplication Prevention: To avoid redundant entries and potential billing issues, we will enforce unique constraints and primary keys to maintain data integrity.

Ethical Considerations in Database Governance
Ethical data handling is critical for maintaining trust and ensuring legal compliance. For our vehicle manufacturing database, ethical issues such as privacy, fairness, and transparency must be prioritized:
• Privacy and Data Protection: We will adhere to GDPR and similar regulations, ensuring lawful data collection and ensuring that customer data is not shared without consent (European Commission, 2018).
• Ethical Use of Customer Data: We will ensure that explicit consent is obtained before using customer data for any marketing or other secondary purposes (Richards & King, 2014).
• Data Retention and Ownership: We will implement data retention policies that define how long customer and vehicle data are kept and ensure that unnecessary records are deleted or anonymized.
• Bias and Discrimination: We will address potential biases in our database and decision-making systems, ensuring fairness and transparency, particularly in AI-driven decisions (O'Neil, 2016).

By ensuring security, integrity, and ethical compliance in our vehicle manufacturing database, we protect sensitive data, maintain business continuity, and build trust with our customers. Through the implementation of best practices for data governance, our database

will remain a reliable and secure resource for managing operations. We must continue to evolve these frameworks to address emerging technological challenges and comply with regulatory standards.

References

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377-387.

Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management.* Pearson.

Crawford, K., & Schultz, J. (2014). Big Data and Due Process: Toward a Framework to Redress Predictive Privacy Harms. *Boston College Law Review, 55*(1), 93-128.

Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems.* Pearson.

European Commission (2018). General Data Protection Regulation (GDPR). Retrieved from https://gdpr.eu/

Floridi, L. (2013). The ethics of information. *Oxford University Press.*

Halfond, W. G., Viegas, J., & Orso, A. (2006). A classification of SQL-injection attacks and countermeasures. *Proceedings of the IEEE International Symposium on Secure Software Engineering, 1*(1), 13-22.

Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287-317.

Richards, N. M., & King, J. H. (2014). Big Data Ethics. *Wake Forest Law Review, 49*, 393-432.

Sandhu, R., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE Computer, 29*(2), 38-47.

Stallings, W. (2019). *Cryptography and Network Security: Principles and Practice.* Pearson.

Appendix

Appendix A - DDL Statement

Statement1

```
CREATE TABLE Retail Customer
    (
     customer_id VARCHAR2 (10)  NOT NULL ,
     first_name  VARCHAR2 (25)  NOT NULL ,
     last_name   VARCHAR2 (25)  NOT NULL ,
     gender      VARCHAR2 (10)  NOT NULL ,
     email       VARCHAR2 (25) ,
     city        VARCHAR2 (25) ,
     county      VARCHAR2 (25) ,
     postcode    VARCHAR2 (10)
    )
```

Table created.

Statement2

```
ALTER TABLE Customer
    ADD CONSTRAINT Customer_PK PRIMARY KEY ( customer_id )
```

Table altered.

Statement3

```
CREATE TABLE Dealer
    (
     dealer_id    VARCHAR2 (10)  NOT NULL ,
     dealer_name  VARCHAR2 (25)  NOT NULL ,
     email        VARCHAR2 (25) ,
     phone_number VARCHAR2 (25) ,
     city         VARCHAR2 (25) ,
     county       VARCHAR2 (25) ,
     postcode     VARCHAR2 (10)
    )
```

Table created.

Statement4

```
ALTER TABLE Dealer
    ADD CONSTRAINT Dealer_PK PRIMARY KEY ( dealer_id )
```

Table altered.

Statement5

```
CREATE TABLE "Order"
    (
     order_id                    VARCHAR2 (10)  NOT NULL ,
     order_date                  DATE  NOT NULL ,
     vehicle_id                  VARCHAR2 (10)  NOT NULL ,
```

```
     customer_id                 VARCHAR2 (10) ,
     dealer_id                   VARCHAR2 (10)  NOT NULL ,
     customer_type               VARCHAR2 (25) ,
     quantity                    NUMBER (10)  NOT NULL ,
     total_amount                NUMBER (25)  NOT NULL ,
     invoice_number              NUMBER (10) ,
     salesperson_id              VARCHAR2 (10) ,
     Customer_customer_id        VARCHAR2 (10) ,
     Sales_person_salesperson_id VARCHAR2 (10) ,
     region_id                   VARCHAR2 (10) ,
     Dealer_dealer_id            VARCHAR2 (10) ,
     Region_region_id            VARCHAR2 (10)  NOT NULL
    )
```

Table created.

Statement6

```
ALTER TABLE "Order"
    ADD CONSTRAINT Order_PK PRIMARY KEY ( order_id )
```

Table altered.

Statement7

```
CREATE TABLE Region
    (
     region_id   VARCHAR2 (10)  NOT NULL ,
     region_name VARCHAR2 (25)  NOT NULL
    )
```

Table created.

Statement8

```
ALTER TABLE Region
    ADD CONSTRAINT Region_PK PRIMARY KEY ( region_id )
```

Table altered.

Statement9

```
CREATE TABLE Sales_person
    (
     salesperson_id   VARCHAR2 (10)  NOT NULL ,
     first_name       VARCHAR2 (25)  NOT NULL ,
     last_name        VARCHAR2 (25)  NOT NULL ,
     region_id        VARCHAR2 (10)  NOT NULL ,
     Region_region_id VARCHAR2 (10)  NOT NULL
    )
```

Table created.

Statement10

```
ALTER TABLE Sales_person
    ADD CONSTRAINT Sales_person_PK PRIMARY KEY ( salesperson_id )
```

Table altered.

Statement11

```
CREATE TABLE Servicing
    (
     servicing_id         VARCHAR2 (10)  NOT NULL ,
     vehicle_id           VARCHAR2 (10)  NOT NULL ,
     customer_id          VARCHAR2 (10)  NOT NULL ,
     dealer_id            VARCHAR2 (10)  NOT NULL ,
     service_type         VARCHAR2 (25)  NOT NULL ,
     fee                  NUMBER ,
     Vehicle_vehicle_id   VARCHAR2 (10)  NOT NULL ,
     Customer_customer_id VARCHAR2 (10) ,
     Dealer_dealer_id     VARCHAR2 (10)
    )
```

Table created.

Statement12

```
ALTER TABLE Servicing
    ADD CONSTRAINT Servicing_PK PRIMARY KEY ( servicing_id )
```

Table altered.

Statement13

```
CREATE TABLE Vehicle
    (
     vehicle_id VARCHAR2 (10)  NOT NULL ,
     name       VARCHAR2 (25)  NOT NULL ,
     type       VARCHAR2 (25) ,
     model      VARCHAR2 (25) ,
     year       DATE ,
     price      NUMBER (25)  NOT NULL ,
     status     VARCHAR2 (25)  NOT NULL
    )
```

Table created.

Statement14

```
ALTER TABLE Vehicle
    ADD CONSTRAINT Vehicle_PK PRIMARY KEY ( vehicle_id )
```

Table altered.

Statement15

```
CREATE TABLE vehicle_order
    (
     Vehicle_vehicle_id VARCHAR2 (10)  NOT NULL ,
     Order_order_id     VARCHAR2 (10)  NOT NULL
    )
```

Table created.

Statement16

```
ALTER TABLE vehicle_order
    ADD CONSTRAINT vehicle_order_PK PRIMARY KEY ( Vehicle_vehicle_id,
Order_order_id )
```

Table altered.

Statement 17

```
ALTER TABLE "Order"
    ADD CONSTRAINT Order_Customer_FK FOREIGN KEY
    (
     Customer_customer_id
    )
    REFERENCES Customer
    (
     customer_id
    )
```

Table altered.

Statement 18

```
ALTER TABLE "Order"
    ADD CONSTRAINT Order_Dealer_FK FOREIGN KEY
    (
     Dealer_dealer_id
    )
    REFERENCES Dealer
    (
     dealer_id
    )
```

Table altered.

Statement 19

```
ALTER TABLE "Order"
    ADD CONSTRAINT Order_Region_FK FOREIGN KEY
    (
     Region_region_id
    )
    REFERENCES Region
    (
     region_id
    )
```

Table altered.

Statement 20

```
ALTER TABLE "Order"
    ADD CONSTRAINT Order_Sales_person_FK FOREIGN KEY
    (
     Sales_person_salesperson_id
    )
    REFERENCES Sales_person
    (
     salesperson_id
    )
```

Table altered.

Statement 21

```
ALTER TABLE Sales_person
    ADD CONSTRAINT Sales_person_Region_FK FOREIGN KEY
    (
     Region_region_id
    )
    REFERENCES Region
    (
     region_id
    )
```

Table altered.

Statement22

```
ALTER TABLE Servicing
    ADD CONSTRAINT Servicing_Customer_FK FOREIGN KEY
    (
     Customer_customer_id
    )
    REFERENCES Customer
    (
     customer_id
    )
```

Table altered.

Statement23

```
ALTER TABLE Servicing
    ADD CONSTRAINT Servicing_Dealer_FK FOREIGN KEY
    (
     Dealer_dealer_id
    )
    REFERENCES Dealer
    (
     dealer_id
    )
```

Table altered.

Statement24

```
ALTER TABLE Servicing
    ADD CONSTRAINT Servicing_Vehicle_FK FOREIGN KEY
    (
     Vehicle_vehicle_id
    )
    REFERENCES Vehicle
    (
     vehicle_id
    )
```

Table altered.

Statement25

```
ALTER TABLE vehicle_order
    ADD CONSTRAINT vehicle_order_Order_FK FOREIGN KEY
    (
     Order_order_id
    )
    REFERENCES "Order"
    (
     order_id
    )
```

Table altered.

Statement26

```
ALTER TABLE vehicle_order
    ADD CONSTRAINT vehicle_order_Vehicle_FK FOREIGN KEY
    (
     Vehicle_vehicle_id
    )
    REFERENCES Vehicle
    (
     vehicle_id
    )
```

Table altered.